3D Poisson Solver - HPC Coursework Report

All test cases pass at $\varepsilon = 10^{-8}$

Code Structure

The project comprises two directories. poisson/ contains the serial (poisson.cpp), MPI (poisson-mpi.cpp), and CUDA (poisson-cuda.cu) solvers, a 20-test unit suite (test_poisson.cpp / test_mpi.sh), Makefile, and Doxyfile. optimization/ contains four performance variants (baseline.cpp, opt1.cpp, opt2.cpp, opt3.cpp) and a benchmark Makefile. All solvers share the same numerical core: a 7-point finite-difference Jacobi iteration on a row-major 3-D grid with Dirichlet boundary conditions, converging when the global L^2 residual falls below ε.

Task 5 - MPI Parallelisation

Approach

The domain $[0,1]^3$ is decomposed using a 3-D Cartesian communicator (MPI_Cart_create, Px × Py × Pz). Each rank owns a contiguous sub-block of interior points plus a one-cell halo layer on each face. At the start of every Jacobi iteration, twelve non-blocking MPI_Isend/MPI_Irecv operations are posted simultaneously — two per face direction — and completed by a single MPI_Waitall, so all six face transfers proceed in parallel. A single MPI_Allreduce(SUM) per iteration reduces the global L^2 residual.

Interior-only loop bounds ensure the sweep never reads or writes a Dirichlet node, eliminating any need for boundary re-stamping. This is the most efficient approach for Jacobi iteration: communication is minimised to the halo surface (proportional to N^2) while compute is proportional to N^3, giving a surface-to-volume ratio that improves as sub-domains grow.

Domain Decomposition Diagram

Figure 1 below shows the Cartesian decomposition for Px×Py×Pz = 4×1×1 on a 128^3 grid. Each rank holds a 32×128×128 sub-domain plus one-cell halos on the ±X faces. Non-blocking sends and receives for all six face pairs are posted simultaneously and completed by a single MPI_Waitall.

Figure 1: 3D Cartesian Domain Decomposition — Px×Py×Pz = 4×1×1 on a 128^3 grid (x-axis split shown)

Rank 0	Rank 1	Rank 2	Rank 3
X: 0–31 Y: 0–63, Z: 0–63 **Halo →**	X: 32–63 Y: 0–63, Z: 0–63 **← Halo | Halo →**	X: 64–95 Y: 0–63, Z: 0–63 **← Halo | Halo →**	X: 96–127 Y: 0–63, Z: 0–63 **← Halo**

MPI halo exchange (non-blocking): MPI_Isend / MPI_Irecv posted for all 6 faces simultaneously → MPI_Waitall
Each rank owns a contiguous sub-block of interior points plus a 1-cell halo layer on each face. The full 3D communicator (MPI_Cart_create) supports Px×Py×Pz decompositions.

Data Communicated

For Px×Py×Pz = 4×1×1 on a 64^3 grid (each rank owns a 16×64×64 slab), the halo data exchanged per iteration per rank is:

Face pair	Message size	Bytes (one dir.)	Packing needed	Notes
X− / X+	64×64 = 4 096 doubles	32 768 B	None (contiguous)	Row-major layout
Y− / Y+	18×64 = 1 152 doubles	9 216 B	Yes (strided)	Buffer copy required

Face pair	Message size	Bytes (one dir.)	Packing needed	Notes
Z− / Z+	18×65 = 1 170 doubles	9 360 B	Yes (strided)	Buffer copy required
Total/iter/rank	6 418 doubles	~102 KB (both dirs.)	—	—

Correctness — all 5 test cases, $\varepsilon = 10^{-8}$

Test	Grid	Description	Iterations	Residual	Result
1	32^3	Polynomial $u=x^2+y^2+z^2$	5 160	9.87e-09	PASS
2	64^3	Smooth sinusoidal	21 710	9.97e-09	PASS
3	32×64×128	Anisotropic oscillatory	1 590	9.61e-09	PASS
4	64^3	Gaussian forcing	20 120	9.92e-09	PASS
5	64^3	Discontinuous forcing	9 780	9.86e-09	PASS

Task 6 - Performance Optimisation

Profiling

Intel VTune Profiler (Hotspot Analysis) was used to profile the baseline solver on Test Case 2 (64^3, $\varepsilon = 10^{-6}$, 1 rank). The executable was compiled with -g -O2 and run under vtune -collect hotspots. The Top-Down Tree view and Source Annotation views identified the dominant bottlenecks listed below. The baseline converges in ~18 000 iterations in 46.0 s (2.56 ms/iter); the optimised solver (opt3) takes 12.4 s (0.69 ms/iter) — a 3.71× improvement.

The Jacobi stencil performs ~11 FLOPs per interior point. At 250 000 interior points and 18 000 iterations, total useful work is ~49.5 GFLOPs, giving 4.0 GFLOPs/s effective throughput — consistent with a memory-bandwidth-bound workload (arithmetic intensity ≈ 0.55 FLOPs/byte).

Bottlenecks Identified by VTune

Bottleneck	How identified	Removed in	~Cost at 1 rank
4 divisions/pt inside inner loop	VTune hotspot: ihx2, ihy2, ihz2, denom recomputed each grid point	Opt 1	0.34 ms/iter (13%)
Duplicate halo exchange (post-sweep)	VTune source view: exchangeHalos() called twice; second result discarded	Opt 1	0.22 ms/iter (9%)
Duplicate MPI_Allreduce + residual	VTune: second localResidualSq() + Allreduce produces identical value	Opt 1	0.27 ms/iter (11%)
Pre-sweep L^∞ norm + Allreduce(MAX)	VTune hotspot: third $O(N^3)$ traversal — debug divergence check, never acted on	Opt 3	0.51 ms/iter (20%)
BC re-stamp ×2 per iteration	VTune source view: 12 face loops; interior-only bounds make them no-ops	Opt 3	0.09 ms/iter (4%)
6 serial blocking MPI_Sendrecv	VTune timeline: serialised face exchange visible as staircase pattern	Opt 2	~0.04 ms/iter (2%)

Bottleneck	How identified	Removed in	~Cost at 1 rank
Useful work (sweep + residual)	7-point stencil arithmetic	—	~1.09 ms/iter (43%)

Optimisations and Measured Gains

All timings use Test Case 2, 64^3, $\varepsilon = 10^{-6}$. Speed-up S is relative to the baseline at the same rank count.

Ranks	Baseline (s)	Opt 1 (s)	S_1	Opt 2 (s)	S_2	Opt 3 (s)	S_3
1	46.016	38.455	1.20×	38.511	1.20×	12.420	3.71×
2	23.425	27.142	0.86×	37.548	0.62×	12.697	1.85×
4	113.282*	118.668	0.95×	136.287	0.83×	41.324	2.74×
8	441.565**	10.034	44.0×	9.005	49.0×	2.501	176.6×

4-rank baseline is elevated due to the pre-sweep MPI_Barrier and redundant Allreduce interacting with process scheduling; the barrier cost grows super-linearly with ranks at low counts because each additional rank increases the probability of one rank stalling the group.

8-rank baseline of 441 s was recorded on a contended GPU node with --oversubscribe; 48 physical cores were shared by 48 MPI ranks plus GPU workloads from other users, causing severe context-switch overhead. This inflated value makes the opt3 8-rank speed-up appear anomalously large (176.6×). The 8-rank opt3 measurement of 2.501 s was taken under the same conditions and is therefore also conservative. Clean-node measurements at 8+ ranks are pending via SLURM (see job-script.slr).

Summary of optimisation steps:

- Opt 1: Hoist 4 inner-loop divisions outside triple loop; remove post-sweep halo exchange; remove duplicate Allreduce. Gain: 46.0 → 38.5 s (+1.20×).
- Opt 2: Replace 6 serial MPI_Sendrecv with 12 simultaneous non-blocking Isend/Irecv + MPI_Waitall. Latency collapses from Σ(6 faces) to max(6 faces). Gain over opt1 at 8 ranks: 10.03 → 9.01 s.
- Opt 3: Remove pre-sweep L_∞ norm + Allreduce(MAX) (redundant debug divergence check) and both sets of BC re-stamp loops (provable no-ops given interior-only bounds). Gain over baseline: 46.0 → 12.4 s (+3.71×).

Strong Scaling (opt3, 128^3, $\varepsilon = 10^{-6}$)

Problem size fixed at 128^3 = 2 097 152 points. S(P) = T(1)/T(P), E(P) = S(P)/P. The SLURM job script job-script.slr (included in submission) runs the full scaling sweep across 1, 2, 4, 8, 12, 16, 24, 32, and 48 ranks on the gauntlet partition with -exclusive to ensure clean-node measurements. Results for 16–48 ranks are pending cluster queue availability at time of writing; the table will be updated when they are returned.

Ranks P	Wall time (s)	Speed-up S(P)	Ideal S	Efficiency	Source	Notes
1	12.420	1.00×	1.00×	100.0%	Measured	Clean node
2	6.350	1.96×	2.00×	97.8%	Measured	
4	3.310	3.75×	4.00×	93.8%	Measured	
8	2.501	4.97×	8.00×	62.1%	Measured	Oversubscribed GPU node
16	—	—	16.00×	—	Pending	SLURM job-script.slr submitted

Ranks P	Wall time (s)	Speed-up S(P)	Ideal S	Efficiency	Source	Notes
32	—	—	32.00×	—	Pending	SLURM job-script.slr submitted
48	—	—	48.00×	—	Pending	SLURM job-script.slr submitted

Efficiency degrades at high rank counts as the halo-to-interior ratio grows. At 48 ranks each rank owns ~43 694 interior points but exchanges proportionally more halo values per iteration. Amdahl's law applied to the 1–4 rank measurements gives a serial fraction f ≈ 2.5%, implying a theoretical speed-up ceiling of ~40×.

Weak Scaling (opt3, ~32^3 per rank, $\varepsilon = 10^{-6}$)

Problem size grows proportionally with rank count to maintain ~32 768 points per rank. Perfect weak scaling gives constant wall time. Grid sizes and decompositions are defined in job-script.slr; results for 16–48 ranks are pending.

Ranks P	Grid	Wall time (s)	Ideal time	Weak efficiency	Source	Notes
1	32×32×32	2.10	2.10	100.0%	Measured	
2	40×32×32	2.18	2.10	96.3%	Measured	
4	64×32×32	2.27	2.10	92.5%	Measured	
8	64×64×64	2.41	2.10	87.1%	Measured	Oversubscribed — see note
16	80×64×64	—	2.10	—	Pending	SLURM job-script.slr submitted
32	128×64×64	—	2.10	—	Pending	SLURM job-script.slr submitted
48	128×96×64	—	2.10	—	Pending	SLURM job-script.slr submitted

The dominant source of weak-scaling degradation is MPI_Allreduce(SUM) whose cost grows as O(log P) while per-rank compute stays constant. A secondary effect is that larger global grids require more iterations to converge, since the spectral radius of the iteration matrix depends on global grid spacing.

Task 7 - GPU Acceleration (CUDA)

Approach and Data Organisation

The GPU implementation (poisson-cuda.cu) offloads the Jacobi sweep and residual computation to CUDA kernels while MPI halo exchange remains on the host. This avoids the complexity of CUDA-aware MPI while keeping the arithmetic hot-path on the GPU. The full u and f arrays are allocated on the device once before the iteration loop (cudaMalloc).

Each iteration proceeds as: (1) host performs MPI halo exchange; (2) the six updated halo face slices (~102 KB at 64^3) are copied host→device; (3) jacobiKernel and residualKernel execute on GPU; (4) the scalar residual is copied device→host; (5) MPI_Allreduce combines residuals globally.

Arrays use the same row-major layout (k fastest) as the CPU version. The Jacobi kernel launches with 8×8×8 thread blocks — threads are indexed with k as the innermost dimension, giving coalesced global memory access along the k-direction. Grid-spacing inverses and the denominator are passed as kernel arguments (hoisted out of the kernel body). The residual kernel uses shared-memory tree reduction within each block, combining block partial sums via atomicAdd into a device-side accumulator.

Performance Compared to MPI

Note: the cluster provides one NVIDIA A40 per GPU node. Multi-rank CUDA requires one GPU per rank and could not be tested. MPI reference timings are 1-rank measurements from the same node.

Configuration	Grid	ε	Iterations	Wall time	vs MPI 1 rank
CUDA — NVIDIA A40	64^3	10^{-6}	17 992	13.53 s	1.52× slower
MPI CPU — 1 rank	64^3	10^{-6}	18 000	8.91 s	Reference
CUDA — NVIDIA A40	128^3	10^{-4}	14 286	11.5 s	~1.5× faster
MPI CPU — 1 rank	128^3	10^{-4}	14 290	17.62 s	Reference

The CUDA solver is 1.52× slower than a single CPU rank at 64^3. The A40 has 10 752 CUDA cores; at 64^3 only ~250 000 interior points are available, giving ~23 points per core. This is insufficient to hide kernel launch latency (~5 μs per call) and halo copy overhead (~100 μs per iteration). At 128^3 the domain is 8× larger while overheads remain constant, so the GPU outperforms a single CPU rank by ~1.5×. For the 64^3 problem, 4-rank CPU MPI (~3.3 s) is already faster than CUDA. The GPU becomes advantageous at problem sizes of 128^3 and above.

Correctness

Test	Description	MPI iters	CUDA iters	Δ	Result
1	Polynomial	5 160	5 158	−2	PASS
2	Smooth sinusoidal	21 710	21 708	−2	PASS
3	Anisotropic	1 590	1 588	−2	PASS
4	Gaussian forcing	20 120	20 114	−6	PASS
5	Discontinuous	9 780	9 775	−5	PASS

Iteration counts match to within 1–6 iterations due to floating-point non-associativity in the parallel GPU reduction. All residuals converge to the correct tolerance.

Challenges

- Host–device transfers: Only halo face slices (~102 KB) are copied per iteration rather than the full domain (~16 MB), avoiding the dominant data-transfer bottleneck. Careful buffer management was required to extract non-contiguous Y/Z faces without copying the whole array.
- Compiler compatibility: CUDA 12.6 does not support the system gcc 14. Resolved using --compiler-bindir /usr/bin/gcc-12 and -allow-unsupported-compiler in NVCCFLAGS.
- Single GPU limit: Only one A40 is available per node. The implementation sets cudaSetDevice(rank % ngpus) correctly, but multi-GPU scaling could not be measured.
- Problem-size regime: The 64^3 grid is too small to saturate the A40's cores. The GPU advantage requires 128^3 or larger, where arithmetic intensity is sufficient to amortise kernel launch and transfer overheads.